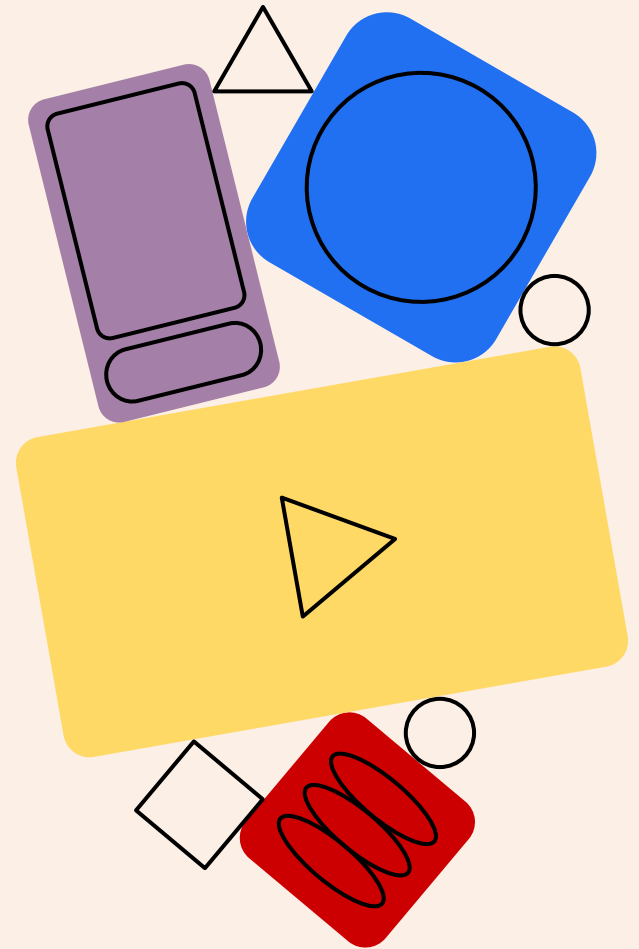


## Data Structures

- Searching
- Sorting



# Outline

- Linear Search
- Binary Search
- Sorting Methods
  - Internal and External Sorting
  - Bubble Sort
  - Quick Sort
  - Merge Sort
  - Insertion Sort

# Introduction - Searching

- **Search** : Locating of a particular element in a data structure.
- **Internal Search** : Searches in which entire table is constantly in main memory are called internal searches.
- **External Search** : Searches in which most of the table is kept in auxiliary storage are called external searches.
- **Storage** : Tables or a file may be organized into an array of records, a linked list, a tree, or even a graph.
- Different search techniques may be suitable for different table organizations, a table is often designed with a specific search techniques.
- The table may be contained completely in memory, completely in auxiliary storage, or it may be divided between the two.

# Introduction - Searching

- **Search Algorithm:** It is an algorithm that accepts an argument  $a$  and tries to find a record whose key is  $a$ .
  - The algorithm may return the entire record or more commonly it may return a pointer to that record.
  - It is possible that the search for a particular argument in a table is unsuccessful; that is; there is no record in the table with argument as its key.
  - A successful search is often called a retrieval.
  - A table of record in which a key is used for retrieval is often called a search table or a dictionary.
- While inserting a record with primary key into a table it is desirable to see whether that record exists or not. This could be done by searching.

# Introduction - Searching

- **Different Searching Methods:**
  - **Sequential Search / Linear Search**
  - **Binary Search**
  - Indexed Sequential Search
  - Interpolation Search
  - Tree Searching
  - Binary Search Tree.

# Linear Search

- In computer science, linear search or sequential search is a method for finding a particular value in a list that consists of checking every one of its elements, one at a time and in sequence, until the desired one is found.
- Linear search is the simplest search algorithm.
- It is a special case of brute-force search.
- Its **worst case cost** is proportional to the **number of elements in the list**.

```
# Input: Array A, integer key
# Output: first index of key in A
# or -1 if not found
Algorithm: Linear_Search
for i = 0 to last index of A:
    if A[i] equals key:
        return i
return -1
```

# Linear Search

Search for **1** in given array

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 9 | 3 | 1 | 8 |
|---|---|---|---|---|

Comparing value of  $i^{\text{th}}$  index with element to be search one by one until we get searched element or end of the array

Step 1:  $i=0$

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 9 | 3 | 1 | 8 |
|---|---|---|---|---|

$\uparrow$   
 $i$

Step 1:  $i=2$

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 9 | 3 | 1 | 8 |
|---|---|---|---|---|

$\uparrow$   
 $i$

Step 1:  $i=1$

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 9 | 3 | 1 | 8 |
|---|---|---|---|---|

$\uparrow$   
 $i$

Step 1:  $i=3$

|   |   |   |          |   |
|---|---|---|----------|---|
| 2 | 9 | 3 | <b>1</b> | 8 |
|---|---|---|----------|---|

$\uparrow$   
 $i$

Element found at  $i^{\text{th}}$  index,  $i=3$

# Linear Search - Efficiency

- Searching a table of constant size  $N$
- The number of comparison depends on where the records with the argument key appears in the table.
- If the record is the first one in the table only one comparison is performed.
- If the record is last one in the table,  $n$  comparisons are necessary.
- If it is equally likely for the argument to appear at any given table position, a successful search will take (on the average)  $(n+1)/2$  comparisons is  $O(n)$ .

# Introduction - Binary Search

- If we have an **array** that is **sorted**, we can use a much more efficient algorithm called a **Binary Search**.
- In binary search **each time** we **divide array** into **two equal half** and **compare middle element** with **search element**.
- Searching Logic
  - If **middle element** is **equal to search element** then we got that element and **return that index**
  - if **middle element** is **less than search element** we **look right part** of array
  - if **middle element** is **greater than search element** we **look left part** of array.

# Binary Search

# Input: Sorted Array A, integer key

# Output: first index of key in A,

# or -1 if not found

**Algorithm: Binary\_Search (A, left, right)**

left = 0, right = n-1

while left < right

    middle = [left + right]/2

    if A[middle] matches key

        return middle

    else if key less than A[middle]

        right = middle - 1

    else

        left = middle + 1

return -1

# Binary Search

Search for **6** in given array

|       |           |   |   |    |    |    |    |    |     |            |
|-------|-----------|---|---|----|----|----|----|----|-----|------------|
|       | -1        | 5 | 6 | 18 | 19 | 25 | 46 | 78 | 102 | 114        |
| Index | 0         | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8   | 9          |
|       | ↑<br>left |   |   |    |    |    |    |    |     | ↑<br>right |

Key=6, No of Elements = 10, so left = 0, right=9

**Step 1:**

**middle index** =  $(\text{left} + \text{right}) / 2 = (0+9)/2 = 4$

**middle element value** =  $a[4] = 19$

**Key=6** is **less than** middle element = **19**, so **right** = middle - 1 = 4 - 1 = **3**, **left** = **0**

|       |           |   |   |            |    |    |    |    |     |     |
|-------|-----------|---|---|------------|----|----|----|----|-----|-----|
|       | -1        | 5 | 6 | 18         | 19 | 25 | 46 | 78 | 102 | 114 |
| Index | 0         | 1 | 2 | 3          | 4  | 5  | 6  | 7  | 8   | 9   |
|       | ↑<br>left |   |   | ↑<br>right |    |    |    |    |     |     |

# Binary Search

**Step 2:**

**middle index** =  $(\text{left} + \text{right}) / 2 = (0+3)/2 = 1$

**middle element value** =  $a[1] = 5$

**Key=6** is **greater than** middle element = **5**, so **left** = middle + 1 = 1 + 1 = **2**, **right** = **3**

|       |    |   |      |       |    |    |    |    |     |     |
|-------|----|---|------|-------|----|----|----|----|-----|-----|
|       | -1 | 5 | 6    | 18    | 19 | 25 | 46 | 78 | 102 | 114 |
| Index | 0  | 1 | 2    | 3     | 4  | 5  | 6  | 7  | 8   | 9   |
|       |    |   | ↑    | ↑     |    |    |    |    |     |     |
|       |    |   | left | right |    |    |    |    |     |     |

**Step 3:**

**middle index** =  $(\text{left} + \text{right}) / 2 = (2+3)/2 = 2$

**middle element value** =  $a[2] = 6$

**Key=6** is **equals to** middle element = **6**, so **element found**

|       |    |   |               |    |    |    |    |    |     |     |
|-------|----|---|---------------|----|----|----|----|----|-----|-----|
|       | -1 | 5 | 6             | 18 | 19 | 25 | 46 | 78 | 102 | 114 |
| Index | 0  | 1 | 2             | 3  | 4  | 5  | 6  | 7  | 8   | 9   |
|       |    |   | ↑             |    |    |    |    |    |     |     |
|       |    |   | Element Found |    |    |    |    |    |     |     |

# Binary Search

Algorithm: Function `biniter(T[1,...,n], x)`

if  $x > T[n]$  then return  $n+1$

$i \leftarrow 1$ ;

$j \leftarrow n$ ;

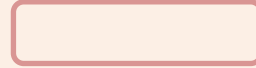
while  $i < j$  do

$k \leftarrow (i + j) \div 2$

if  $x \leq T[k]$  then  $j \leftarrow k-1$

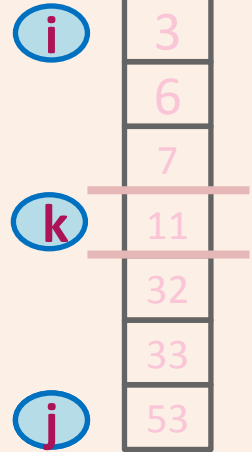
else  $i \leftarrow k + 1$

return  $i$



$n = 7$

$x = 33$



# Binary Search- Time Complexity

- The time complexity of binary search
  - For successful search:
    - worst case is  $O(\log n)$
    - Best case is  $O(1)$
    - For unsuccessful search,  $O(n)$
- For unsuccessful search:
  - worst case, best case and average case,  $O(\log n)$

# Introduction - Sorting

- Sorting is the operation of arranging the records of a table into some sequential order according to ordering criteria.
- The term sorting means arranging the elements of the array so that they are placed in some relevant order which may either be ascending order or descending order. That is, if  $A$  is an array then the elements of  $A$  are arranged in sorted order (ascending order) in such a way that,  $A[0] < A[1] < A[2] < \dots < A[N]$
- For example, if we have an array that is declared and initialized as, `int A[] = {21, 34, 11, 9, 1, 0, 22};`
- Then the sorted array (ascending order) can be given as, `A[] = {0, 1, 9, 11, 21, 22, 34}`

# Introduction - Sorting

- A sorting algorithm is defined as an algorithm that puts elements of a list in a certain order (that can either be numerical order, lexicographical order or any user-defined order).
- Efficient sorting algorithms are widely used to optimize the use of other algorithms like search and merge algorithms which require sorted lists to work correctly.
- There are two types of sorting:
  - **Internal** sorting which deals with sorting the data stored in computer's memory
  - **External** sorting which deals with sorting the data stored in files. External sorting is applied when there is voluminous data that cannot be stored in computer's memory.
- **Sorting** arranges data in a sequence which makes searching easier.

# Introduction - Sorting

- There are many types of sorting. We will be learning about:
  - Bubble Sort
  - Insertion Sort
  - Quick Sort
  - Merge Sort

# Bubble Sort

- Here two records are interchanged immediately upon discovering that they are out of order
- During the **first pass R1 and R2 are compared and interchanged in case of out of order**, this process is **repeated for records R2 and R3, and so on**.
- This method will cause records with **small key to move “bubble up”**
- **After the first pass**, the record with **largest key will be in the nth position**.
- On each successive pass, the records with the next largest key will be placed in position  $n-1$ ,  $n-2$  ....., 2 respectively
- This approach required at most  $n-1$  passes, The complexity of bubble sort is  $O(n^2)$

# Bubble Sort

## Unsorted Array

|    |    |    |    |    |
|----|----|----|----|----|
| 45 | 34 | 56 | 23 | 12 |
|----|----|----|----|----|

Pass 1 :

|    |
|----|
| 34 |
| 45 |
| 56 |
| 23 |
| 12 |

swap

|    |
|----|
| 34 |
| 45 |
| 56 |
| 23 |
| 12 |

|    |
|----|
| 34 |
| 45 |
| 23 |
| 56 |
| 12 |

swap

|    |
|----|
| 34 |
| 45 |
| 23 |
| 12 |
| 56 |

swap

# Bubble Sort

Pass 2 :

|    |
|----|
| 34 |
| 45 |
| 23 |
| 12 |
| 56 |

|    |
|----|
| 34 |
| 23 |
| 45 |
| 12 |
| 56 |



swap

|    |
|----|
| 34 |
| 23 |
| 12 |
| 45 |
| 56 |



swap

Pass 3 :

|    |
|----|
| 23 |
| 34 |
| 12 |
| 45 |
| 56 |



swap

|    |
|----|
| 23 |
| 12 |
| 34 |
| 45 |
| 56 |



swap

Pass 4 :

|    |
|----|
| 12 |
| 23 |
| 34 |
| 45 |
| 56 |



swap

# Bubble Sort

- Given a vector **K** of **N** elements
- This procedure **rearrange** the **vector** in **ascending order** using **Bubble Sort**
- The variable **PASS & LAST** denotes the **pass index** and position of the first element in the vector
- The variable **EXCHS** is used to count number of exchanges made on any pass
- The variable **I** is used to index elements

# Bubble Sort

## 1. [Initialize]

LAST  $\leftarrow$  N

## 2. [Loop on pass index]

Repeat thru step 5 for PASS = 1, 2, 3, .... , N-1

## 3. [Initialize exchange counter for this pass]

EXCHS  $\leftarrow$  0

## 4. [Perform pairwise comparisons on unsorted elements]

Repeat for I = 1, 2, ....., LAST - 1

IF  $K[I] > K[I+1]$

Then  $K[I] \leftrightarrow K[I+1]$

EXCHS  $\leftarrow$  EXCHS + 1

## 5. [Any exchange made in this pass?]

IF EXCHS = 0

Then Return (Vector is sorted, early return)

ELSE LAST  $\leftarrow$  LAST - 1

## 6. [Finished]

Return

# Insertion Sort

- In insertion sort, **every iteration moves** an **element** from **unsorted portion** to **sorted portion** until all the elements are sorted in the list
- **Steps for Insertion Sort**
  - Assume that **first element** in the list is in **sorted portion** of the list and **remaining all elements** are in **unsorted portion**.
  - Select **first element** from the **unsorted list** and **insert** that element **into the sorted list** in **order specified**.
  - **Repeat** the above process **until all** the **elements** from the **unsorted list** are **moved into the sorted list**.
- This algorithm is not suitable for large data sets

# Insertion Sort

- To sort a **unsorted list** with 'n' number of **elements** we need to make  $(1+2+3+.....+n-1) = (n(n-1))/2$  number of **comparisons** in the worst case.
- If the list **already sorted**, then it requires 'n' number of **comparisons**.
- Worst Case :  $\Theta(n^2)$
- Best Case :  $\Omega(n)$
- Average Case :  $\Theta(n^2)$

# Insertion Sort Example

Sort given array using Insertion Sort

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 6 | 5 | 3 | 1 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

Pass - 1 : Select First Record and considered as Sorter Sub-array

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 6 | 5 | 3 | 1 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

Sorted | Unsorted

Pass - 2 : Select Second Record and Insert at proper place in sorted array

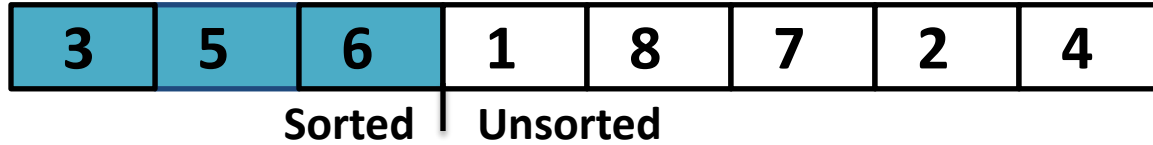
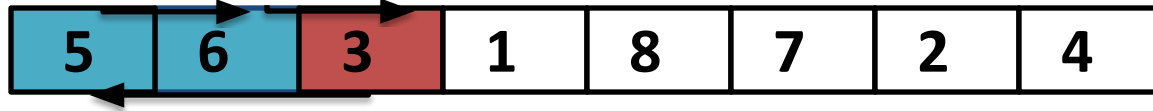
|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 6 | 5 | 3 | 1 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 5 | 6 | 3 | 1 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

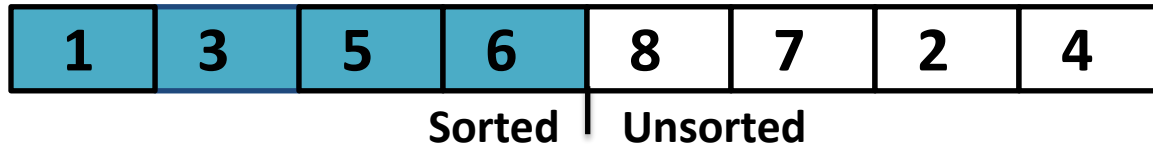
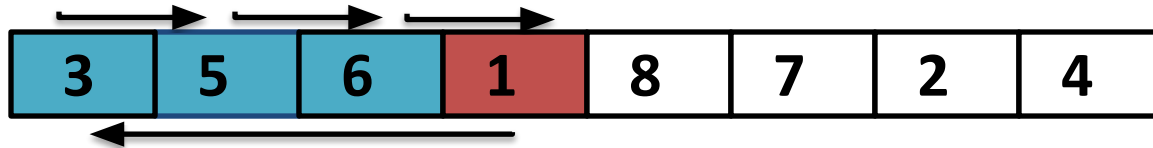
Sorted | Unsorted

## Insertion Sort Example

Pass - 3 : Select Third record and Insert at proper place in sorted array



Pass - 4 : Select Forth record and Insert at proper place in sorted array



## Insertion Sort Example

Pass - 5 : Select Fifth record and Insert at proper place in sorted array

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 5 | 6 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

8 is at proper position

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 5 | 6 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

Sorted | Unsorted

Pass - 6 : Select Sixth Record and Insert at proper place in sorted array

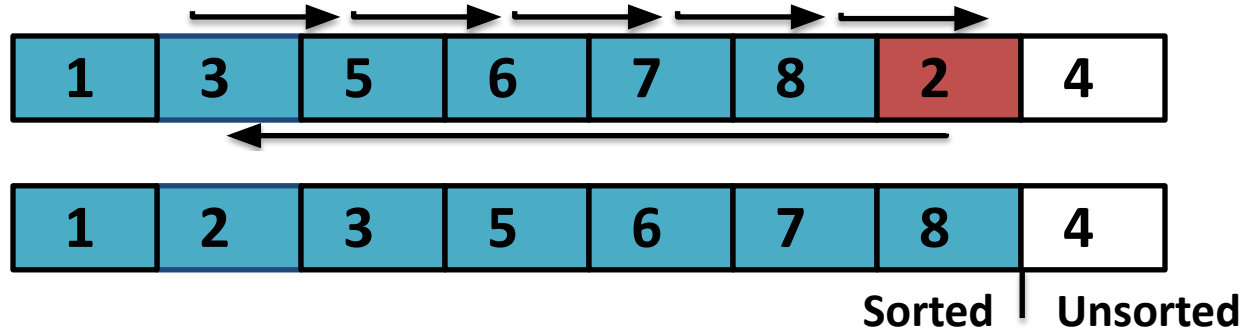
|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 5 | 6 | 8 | 7 | 2 | 4 |
|---|---|---|---|---|---|---|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 5 | 6 | 7 | 8 | 2 | 4 |
|---|---|---|---|---|---|---|---|

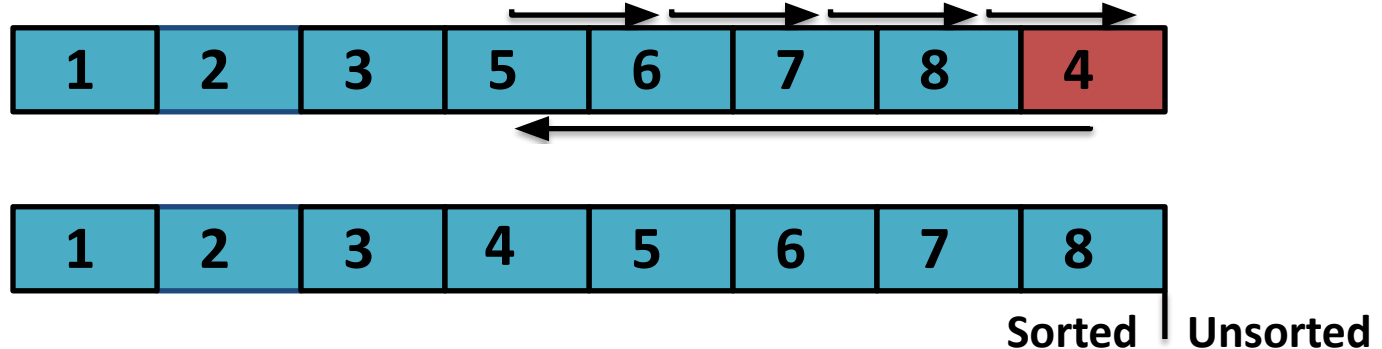
Sorted | Unsorted

## Insertion Sort Example

Pass - 7 : Select Seventh record and Insert at proper place in sorted array



Pass - 8 : Select Eighth Record and Insert at proper place in sorted array



# Insertion Sort

- Insertion sort (ARR, N) where ARR is an array of N elements  
Step 1: Repeat Steps 2 to 5 for K = 1 to N  
Step 2: SET TEMP = ARR[K]  
Step 3: SET J = K - 1  
Step 4: Repeat while TEMP <= ARR[J]  
          SET ARR[J + 1] = ARR[J]  
          SET J = J - 1  
          [END OF INNER LOOP]  
Step 5: SET ARR[J + 1] = TEMP  
          [END OF LOOP]  
Step 6: EXIT
- Bubble sort, selection sort, and insertion sort are all  $O(n^2)$

# Insertion Sort

- Insertion sort (ARR, N) where ARR is an array of N elements

Step 1: Repeat Steps 2 to 5 for  $K = 1$  to  $N$

Step 2: SET  $TEMP = ARR[K]$

Step 3: SET  $J = K - 1$

Step 4: Repeat while  $TEMP \leq ARR[J]$

    SET  $ARR[J + 1] = ARR[J]$

    SET  $J = J - 1$

    [END OF INNER LOOP]

Step 5: SET  $ARR[J + 1] = TEMP$

    [END OF LOOP]

Step 6: EXIT

- Bubble sort and insertion sort are all  $O(n^2)$
- Insertion Sort can be used for small arrays.

# Divide and Conquer

- Divide and Conquer cuts the problem in half each time, but uses the result of both halves:
  - cut the problem in half until the problem is trivial
  - solve for both halves
  - combine the solutions
- 
-

# Divide and Conquer

- Base case
  - single element ( $n=1$ ), return
- Divide A into two subarrays: FirstPart, SecondPart
  - Two Subproblems:
    - sort the FirstPart
    - sort the SecondPart
  - Recursively
    - sort FirstPart
    - sort SecondPart
- Combine sorted FirstPart and sorted second part

# Merge Sort

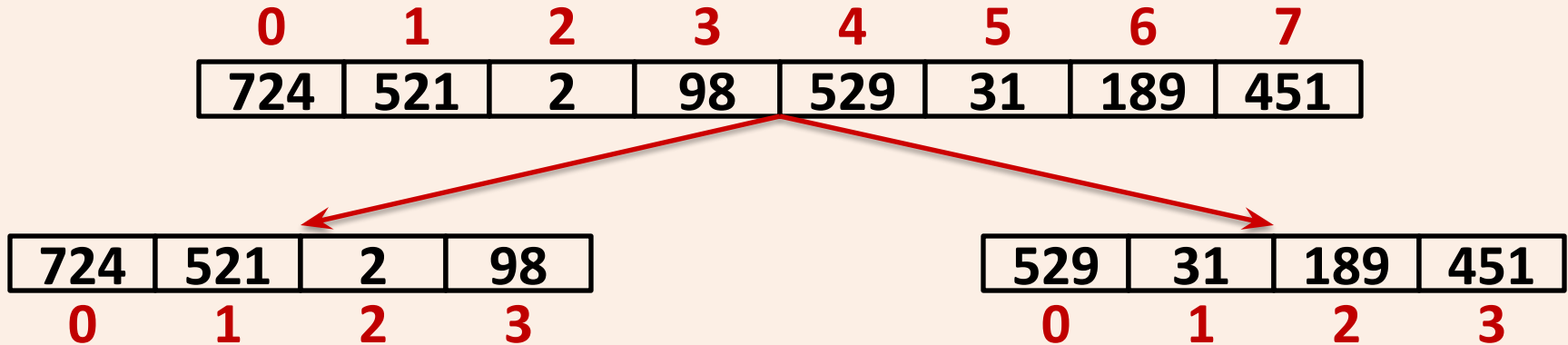
- The **operation of sorting** is closely related to **process of merging**
- Merge Sort is a **divide and conquer algorithm**
- It is based on the **idea of breaking down a list into several sub-lists** until each sub list consists of a **single element**
- **Merging those sub lists** in a manner that results into a sorted list
- **Procedure**
  - **Divide** the unsorted **list into N sub** lists, **each containing 1 element**
  - Take **adjacent pairs** of two singleton lists and **merge them** to form a **list of 2 elements**. N will now convert into  $N/2$  list of size 2
  - Repeat the process till a single sorted list is obtained
- Time complexity is  **$O(n \log n)$**
- **By John Von Neumann, 1945 (First Divide and Conquer)**

# Merge Sort

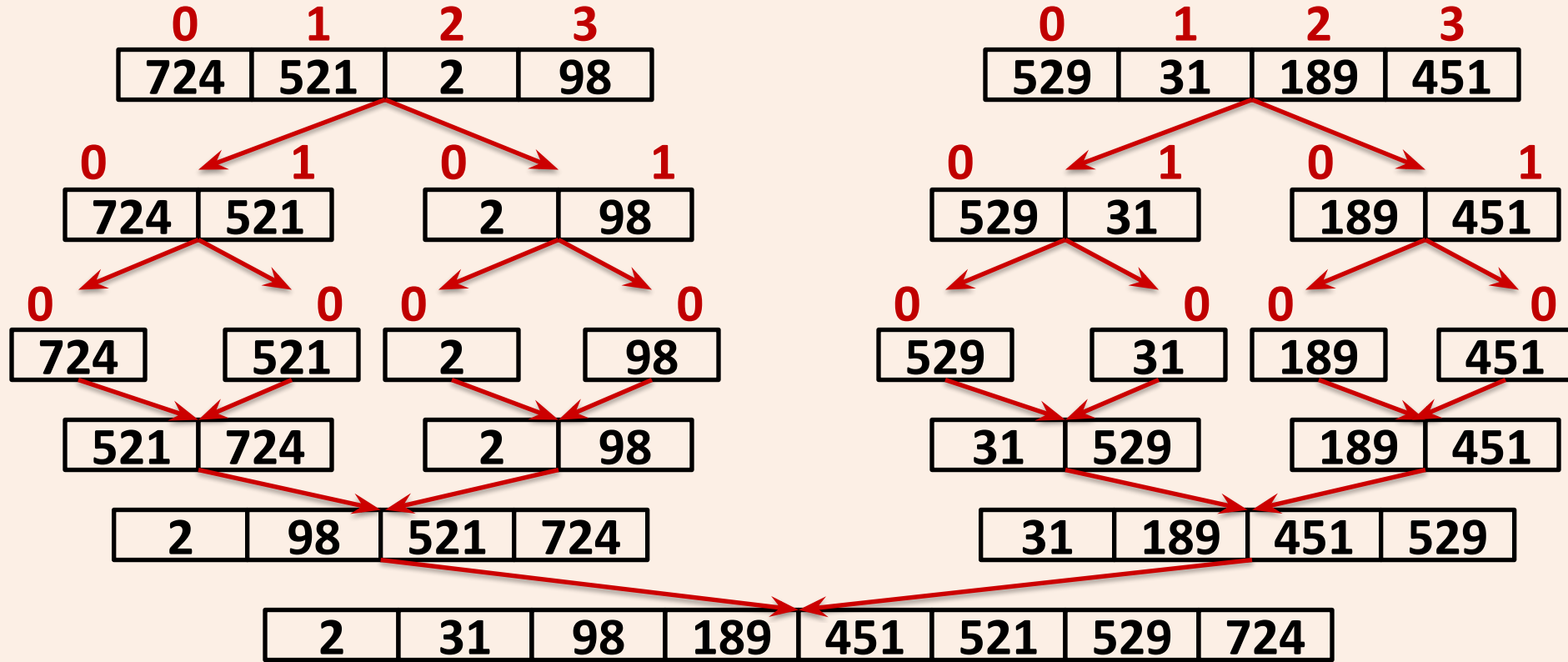
## Unsorted Array

|     |     |   |    |     |    |     |     |
|-----|-----|---|----|-----|----|-----|-----|
| 724 | 521 | 2 | 98 | 529 | 31 | 189 | 451 |
| 0   | 1   | 2 | 3  | 4   | 5  | 6   | 7   |

Step 1: Split the selected array (as evenly as possible)



# Merge Sort



Step: Select the subarray, Split the selected array (as evenly as possible)

# Merge Sort

## **Mergesort(Passed an array)**

if array size  $> 1$

Divide array in half

Call Mergesort on first half.

Call Mergesort on second half.

Merge two halves.

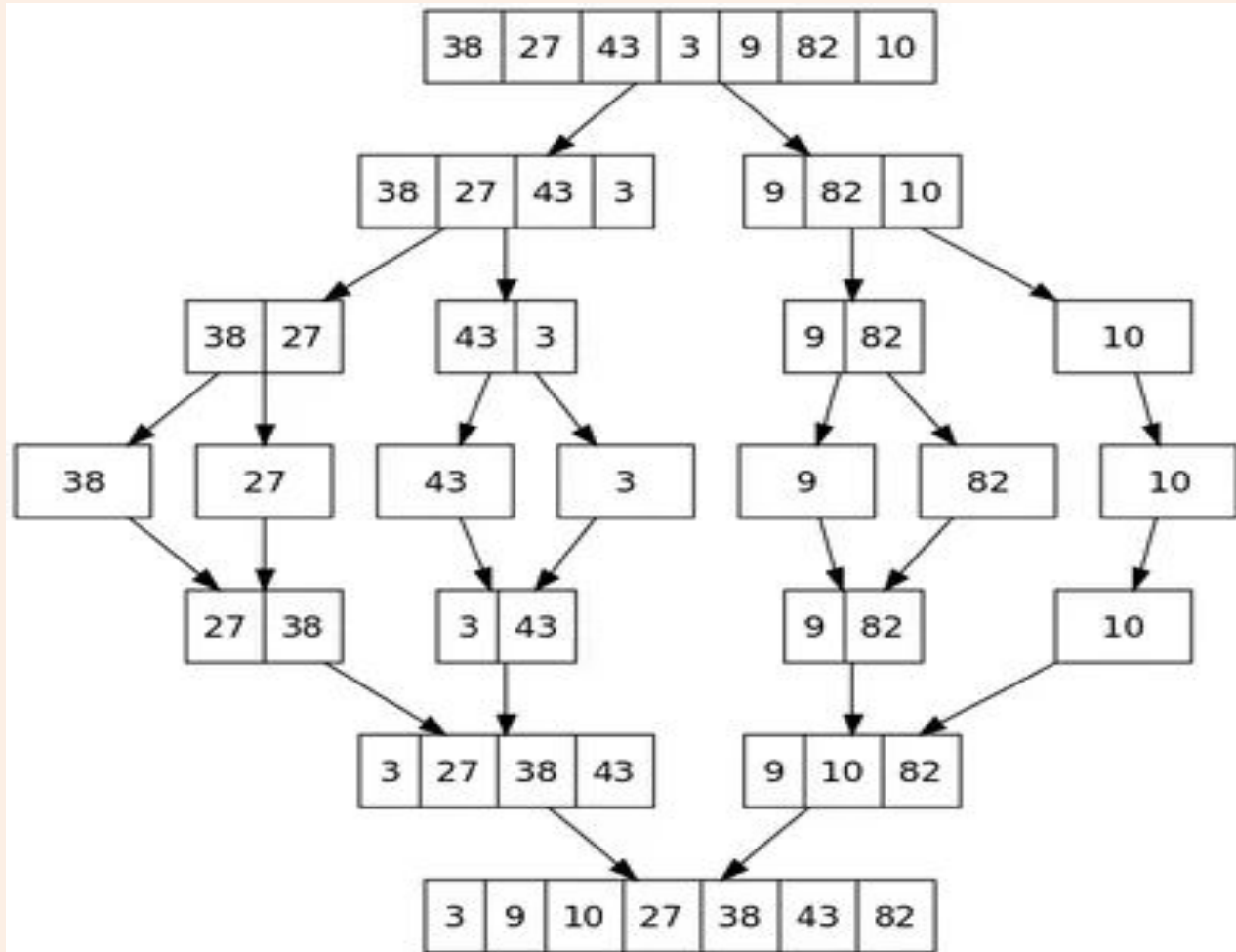
## **Merge(Passed two arrays)**

Compare leading element in each array

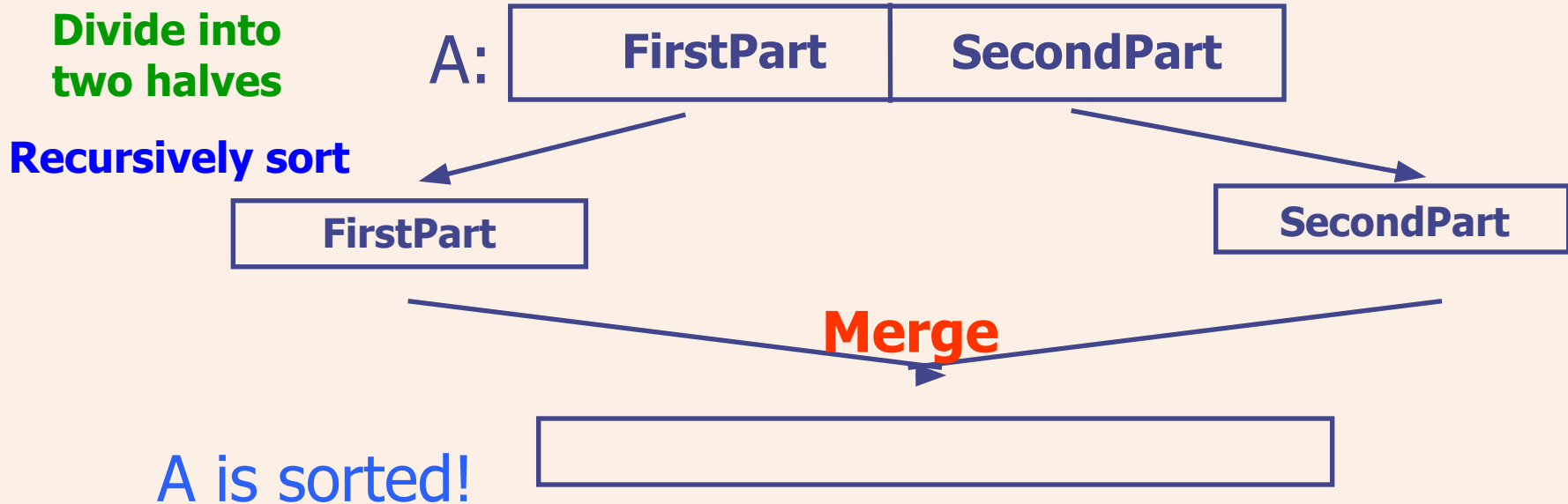
Select lower and place in new array.

(If one input array is empty then place remainder of other array in output array)

# Merge Sort

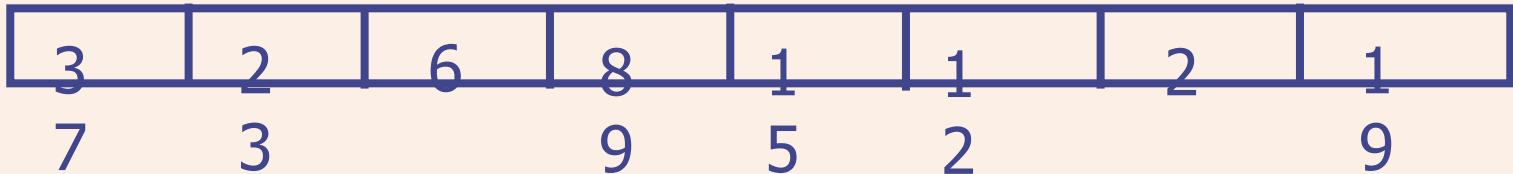


# Merge Sort: Idea



# Merge sort

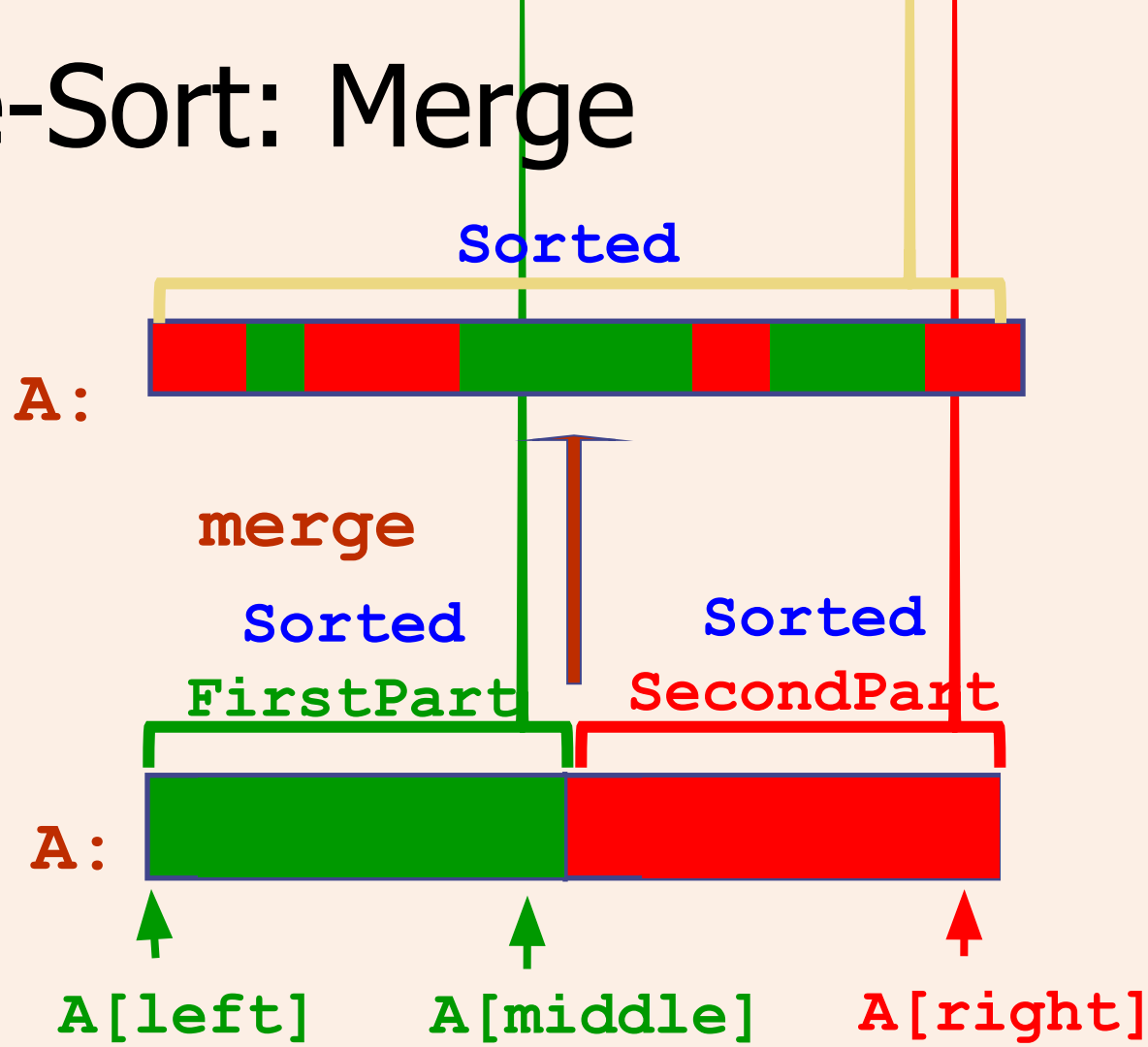
- A divide-and-conquer algorithm:
- Divide the unsorted array into 2 halves until the sub-arrays only contain one element
- Merge the sub-problem solutions together:
  - Compare the sub-array's first elements
  - Remove the smallest element and put it into the result array
  - Continue the process until all elements have been put into the result array



# Merge working

- Merge working
  - Access the first item from both sequences
  - While not finished with either sequence
    - Compare the current items from the two sequences, copy the smaller current item to the output sequence, and access the next item from the input sequence whose item was copied
  - Copy any remaining items from the first sequence to the output sequence
  - Copy any remaining items from the second sequence to the output sequence

# Merge-Sort: Merge



## Mergesort(Passed an array)

if array size > 1

Divide array in half

Call Mergesort on first half.

Call Mergesort on second half.

Merge two halves.

## Merge(Passed two arrays)

Compare leading element in each array

Select lower and place in new array.

(If one input array is empty then place  
remainder of other array in output array)

# Recursive merge sort Algorithm

merge\_sort ( a, start, end)

Step 1:

if start < end

(i) mid = (start+end)/2;

(ii) merge\_sort(a,start,mid);

(iii) merge\_sort(a,mid+1,end);

(iv) merge(a,start,mid,end);

Step 2: Return

# Merge algorithm(a,beg,mid,end)

Result[] is for final sorted array

K is index for result array

Step 1 :Step1: [Initialize]

i=start      j=mid+1      k=0

Step 2: Repeat setp 4 while ((i<=mid) and (j<=end))

Step 3:    if (list\_a[i] < list\_a[j])

    (i) result[k]=list\_a[i]

    (ii) i=i+1

    else if (list\_a[i] > list\_a[j])

        (iv) result[k]=list\_a[j]

        (v) j=j+1

    else // elements of both the list are equal and insert only one of them in result[]

        (vii) result[k]=list\_a[i]    (viii) i = i+1

        (ix) j= j+1

End of IF-ELSE

Step 4 :    k = k+1

Step 5: if( $i > \text{mid}$ ) [copy the element of right su-array]

Step 6: Repeat through step 6 for  $j \leq \text{End}$

(i)  $\text{result}[k] = \text{list\_a}[j]$

(ii)  $j = j + 1$

(iii)  $k = k + 1$

Step 7: [copy of element of left subarray if left subarray are still reamining]

else

Step 8: Repeat through step 8 for  $i \leq \text{mid}$

(i)  $\text{result}[k] = \text{list\_a}[i]$

(ii)  $i = i + 1$

(iii)  $k = k + 1$

Step 9: Return

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|



Merge



Merge



Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|



Merge



Merge



Merge



Merge



Merge



|       |
|-------|
| Merge |
|-------|



Merge



|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |
|---|
| 6 |
|---|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |
|----|
| 33 |
|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|       |
|-------|
| Merge |
|-------|



Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |
|---|----|
| 6 | 33 |
|---|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |
|---|----|----|
| 6 | 33 | 42 |
|---|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

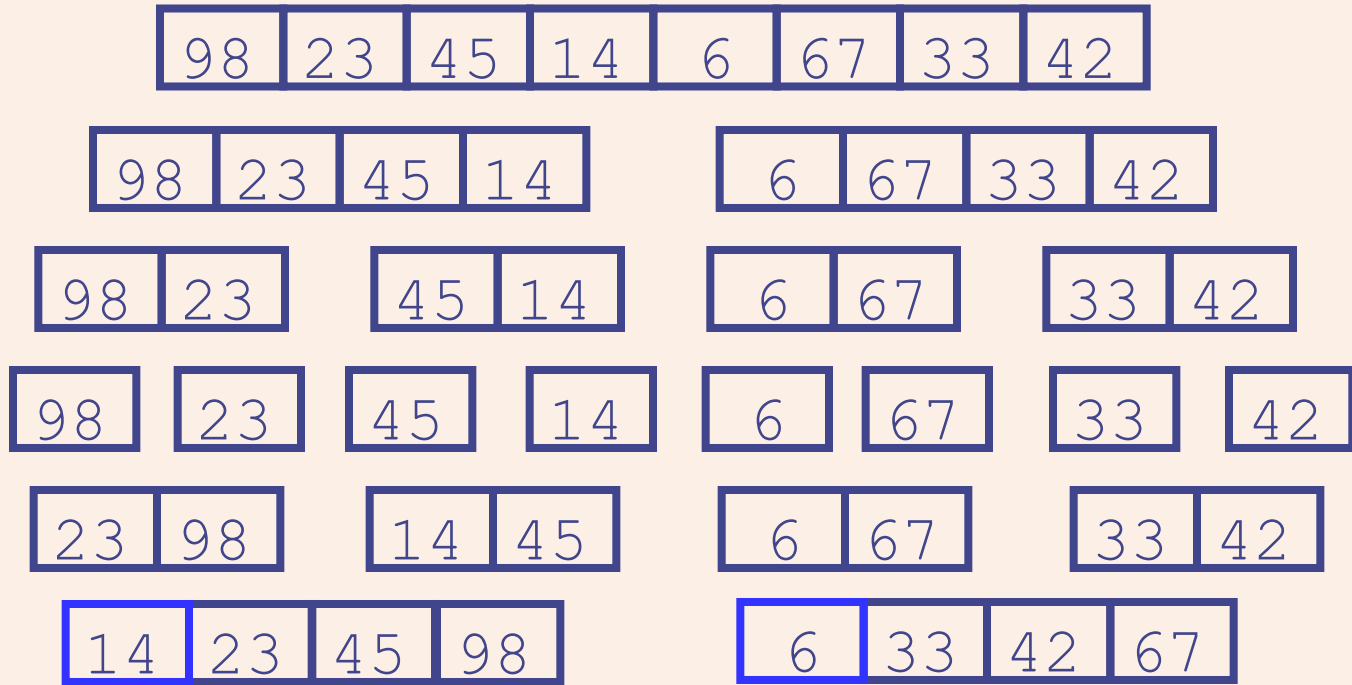
|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

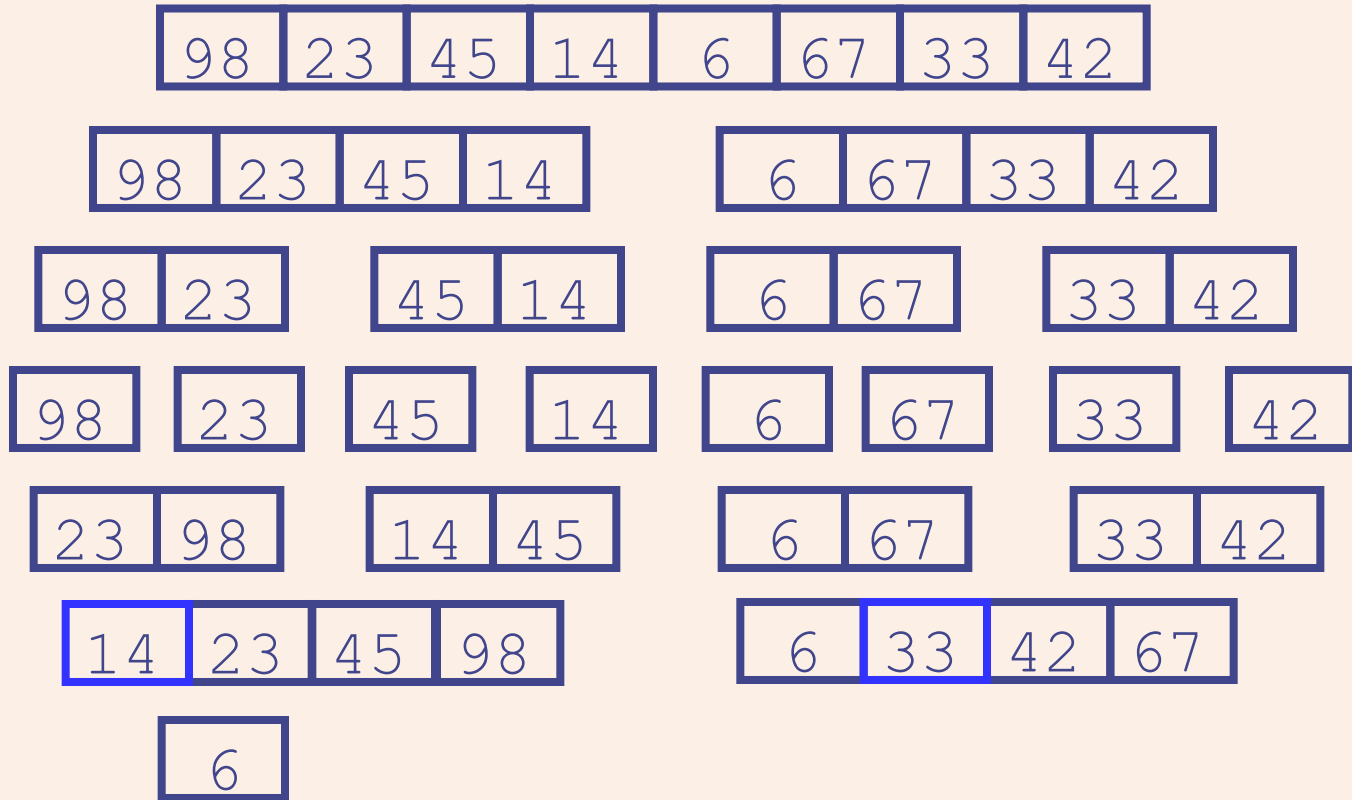
|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|       |
|-------|
| Merge |
|-------|



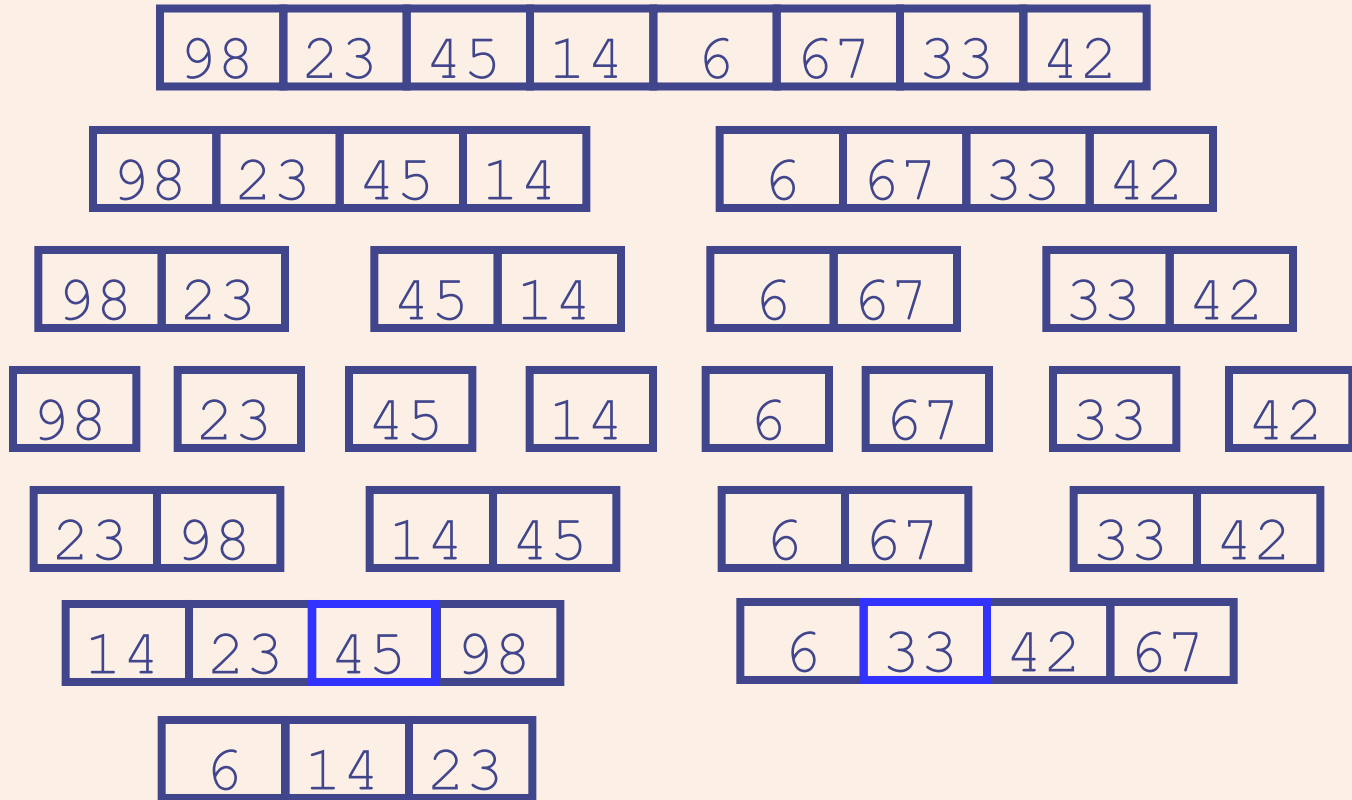
Merge



Merge



Merge



Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 14 | 23 | 33 |
|---|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|   |    |    |    |    |
|---|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 |
|---|----|----|----|----|

|       |
|-------|
| Merge |
|-------|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|   |    |    |    |    |    |
|---|----|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 | 45 |
|---|----|----|----|----|----|

Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|   |    |    |    |    |    |    |
|---|----|----|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 | 45 | 67 |
|---|----|----|----|----|----|----|

Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 | 45 | 67 | 98 |
|---|----|----|----|----|----|----|----|

Merge

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 98 | 23 | 45 | 14 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 67 | 33 | 42 |
|---|----|----|----|

|    |    |
|----|----|
| 98 | 23 |
|----|----|

|    |    |
|----|----|
| 45 | 14 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |
|----|
| 98 |
|----|

|    |
|----|
| 23 |
|----|

|    |
|----|
| 45 |
|----|

|    |
|----|
| 14 |
|----|

|   |
|---|
| 6 |
|---|

|    |
|----|
| 67 |
|----|

|    |
|----|
| 33 |
|----|

|    |
|----|
| 42 |
|----|

|    |    |
|----|----|
| 23 | 98 |
|----|----|

|    |    |
|----|----|
| 14 | 45 |
|----|----|

|   |    |
|---|----|
| 6 | 67 |
|---|----|

|    |    |
|----|----|
| 33 | 42 |
|----|----|

|    |    |    |    |
|----|----|----|----|
| 14 | 23 | 45 | 98 |
|----|----|----|----|

|   |    |    |    |
|---|----|----|----|
| 6 | 33 | 42 | 67 |
|---|----|----|----|

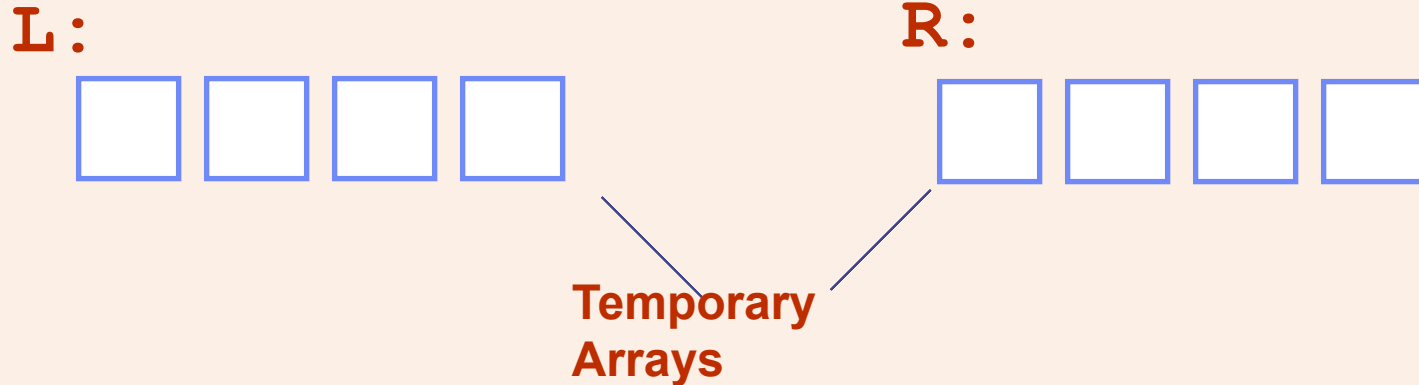
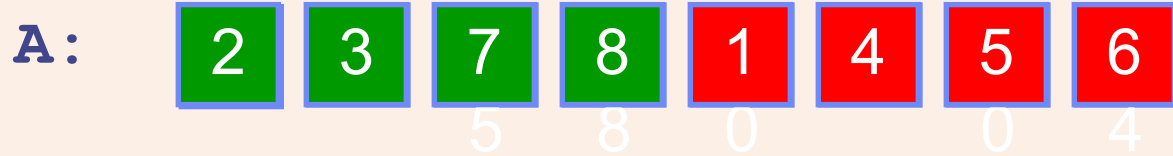
|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 | 45 | 67 | 98 |
|---|----|----|----|----|----|----|----|

|    |    |    |    |   |    |    |    |
|----|----|----|----|---|----|----|----|
| 98 | 23 | 45 | 14 | 6 | 67 | 33 | 42 |
|----|----|----|----|---|----|----|----|



|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 6 | 14 | 23 | 33 | 42 | 45 | 67 | 98 |
|---|----|----|----|----|----|----|----|

# Merge-Sort: Merge Example



# Merge-Sort: Merge Example

A: 

L: 

R: 

# Merge-Sort: Merge Example

A:



k=0

L:



i=0

R:



j=0

# Merge-Sort: Merge Example

A:



k=1

L:



i=1

R:



j=0

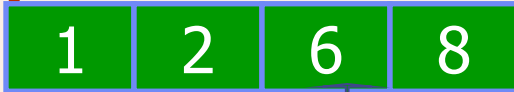
# Merge-Sort: Merge Example

A:



k=2

L:



i=2

R:



j=0

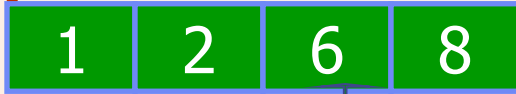
# Merge-Sort: Merge Example

A:



k=3

L:



i=2

R:



j=1

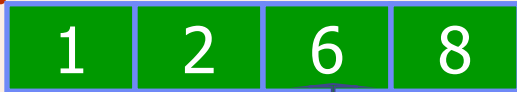
# Merge-Sort: Merge Example

A:



k=4

L:



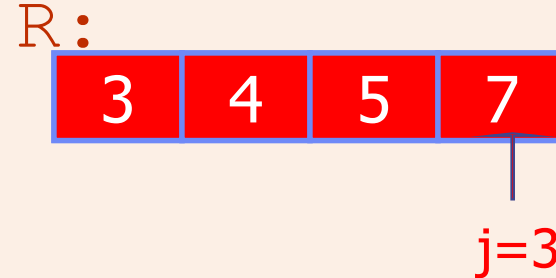
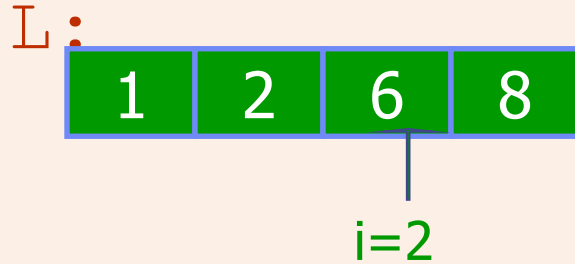
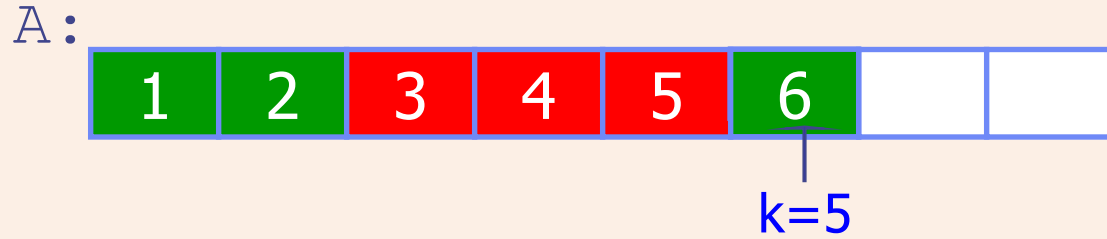
i=2

R:



j=2

# Merge-Sort: Merge Example



# Merge-Sort: Merge Example

A:



k=6

L:



i=3

R:



j=3

# Merge-Sort: Merge Example

A:



k=7

L:



i=3

R:



j=4

# Merge-Sort: Merge Example

A:



↑  
k=8

L:



↑  
i=4

R:



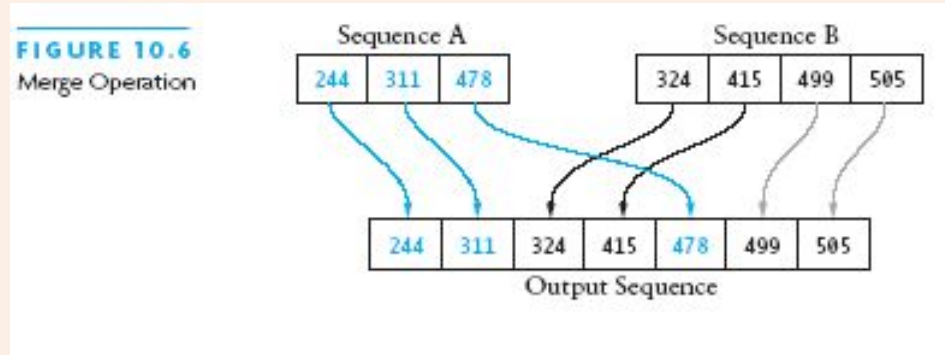
↑  
j=4

# Summary

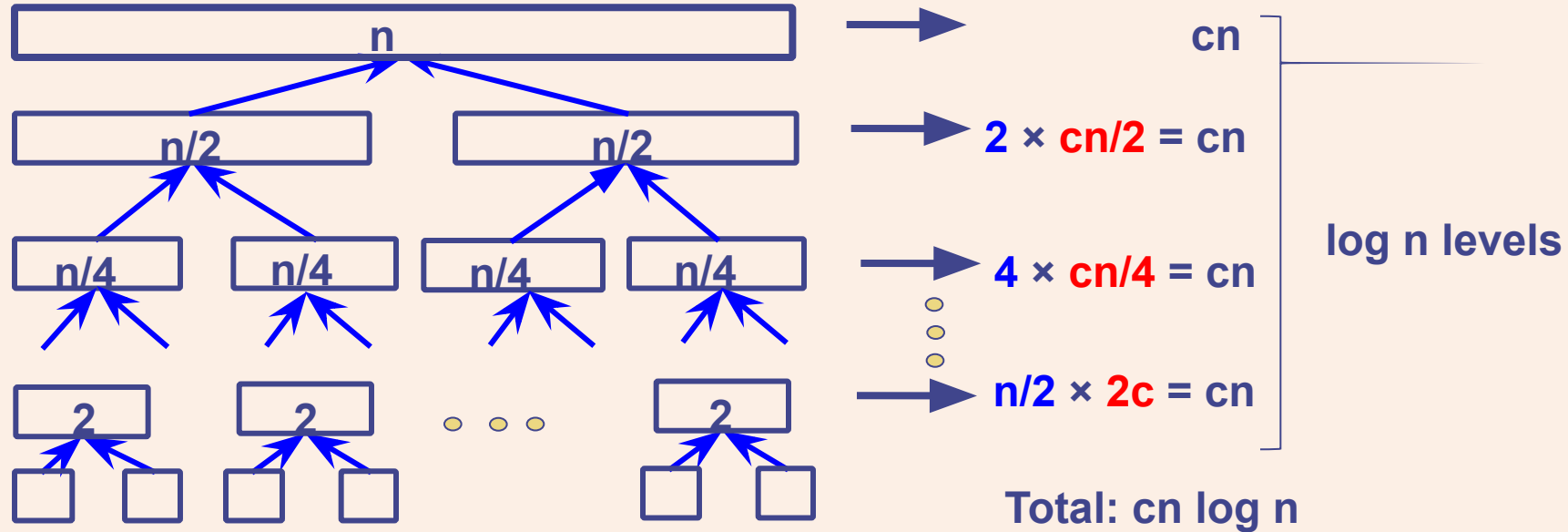
- Divide the unsorted collection into two
- Until the sub-arrays only contain one element
- Then merge the sub-problem solutions together

# Analysis of Merge

- For two input sequences that contain a total of  $n$  elements, we need to move each element's input sequence to its output sequence
  - Merge time is  $O(n)$
- We need to be able to store both initial sequences and the output sequence
  - The array cannot be merged in place
  - Additional space usage is  $O(n)$



# Merge-Sort Analysis



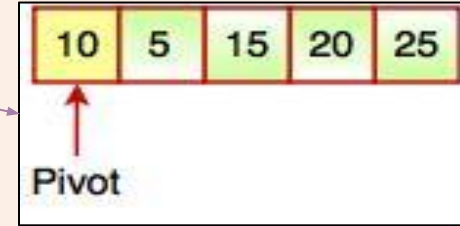
- Total running time:  $\Theta(n \log n)$
- Total Space:  $\Theta(n)$

# Quick Sort

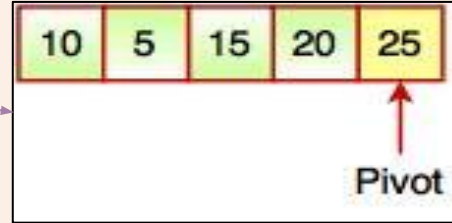
- Quick Sort /Partition Exchange Sort
- **Quick sort** is a highly efficient sorting algorithm and is based on **partitioning of array** of data into **smaller arrays**.
- Quick Sort is **divide and conquer** algorithm.
- At each step of the method, the goal is to place a particular record in its final position within the table.
- In doing so all the records which precedes this record will have smaller keys, while all records that follows it have larger keys.
- This particular records is **termed pivot element**.
- The same process can then be applied to each of these subtables and repeated until all records are placed in their positions.
- **Tony Hoare, 1961 (null pointer)**

# Quick Sort

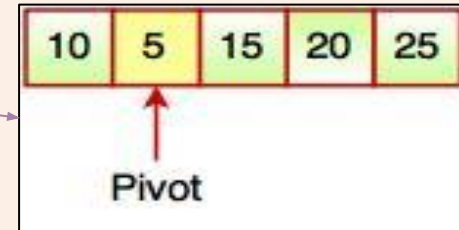
- There are many **different versions** of Quick Sort **that pick pivot** in different ways.
  - Always pick **first element as pivot**. (in our case we have consider this version).



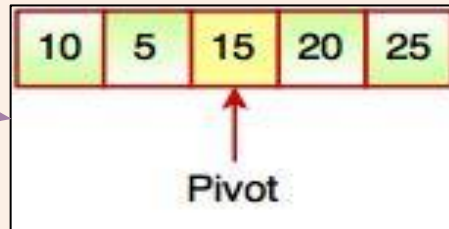
- Always pick **last element as pivot**



- Pick a **random element as pivot**.



- Pick **median as pivot**.



# Quick Sort

- Quick sort partitions an array and then calls itself recursively twice to sort the two resulting sub arrays.
- This algorithm is quite **efficient for large-sized data sets**
- Its average and **worst case complexity** are of  **$O(n^2)$** , where n is the number of items.

# Quick Sort

- In Quick sort algorithm, partitioning of the list is performed using following steps...
  - Consider the first element of the list as **pivot** (i.e., Element at first position in the list).
  - Define two variables  $i$  and  $j$ . Set  $i$  and  $j$  to first and last elements of the list respectively.
  - Increment  $i$  until  $\text{list}[i] > \text{pivot}$  then stop.
  - Decrement  $j$  until  $\text{list}[j] < \text{pivot}$  then stop.
  - If  $i < j$  then exchange  $\text{list}[i]$  and  $\text{list}[j]$ .
  - Repeat steps 3,4 & 5 until  $i > j$ .
  - Exchange the pivot element with  $\text{list}[j]$  element.

# PARTITION (ARR, BEG, END, LOC)

Step 1: [INITIALIZE] SET LEFT = BEG, RIGHT = END, LOC = BEG, FLAG = 0

Step 2: Repeat Steps 3 to 6 while FLAG = 0

Step 3: Repeat while ARR[LOC] <= ARR[RIGHT] AND LOC != RIGHT

SET RIGHT = RIGHT - 1

[END OF LOOP]

Step 4: IF LOC = RIGHT

SET FLAG = 1

ELSE IF ARR[LOC] > ARR[RIGHT]

SWAP ARR[LOC] with ARR[RIGHT]

SET LOC = RIGHT

[END OF IF]

Step 5: IF FLAG = 0

Repeat while ARR[LOC] >= ARR[LEFT] AND LOC != LEFT

SET LEFT = LEFT + 1

[END OF LOOP]

Step 6: IF LOC = LEFT

SET FLAG = 1

ELSE IF ARR[LOC] < ARR[LEFT]

SWAP ARR[LOC] with ARR[LEFT]

SET LOC = LEFT

[END OF IF]

[END OF IF]

Step 7: [END OF LOOP]

Step 8: END

QUICK\_SORT (ARR, BEG, END)

Step 1: IF (BEG < END)

CALL PARTITION (ARR, BEG, END, LOC)

CALL QUICKSORT(ARR, BEG, LOC - 1)

CALL QUICKSORT(ARR, LOC + 1, END)

[END OF IF]

Step 2: END

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 27 | 10 | 36 | 18 | 25 | 45 |
|----|----|----|----|----|----|

We choose the first element as the pivot.  
Set  $loc = 0$ ,  $left = 0$ , and  $right = 5$ .

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 27 | 10 | 36 | 18 | 25 | 45 |
|----|----|----|----|----|----|

$loc$   $right$   
 $left$

Scan from right to left. Since  $a[loc] < a[right]$ , decrease the value of  $right$ .

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 27 | 10 | 36 | 18 | 25 | 45 |
|----|----|----|----|----|----|

$loc$   $right$   
 $left$

Start scanning from left to right. Since  $a[loc] > a[left]$ , increment the value of  $left$ .

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 25 | 10 | 36 | 18 | 27 | 45 |
|----|----|----|----|----|----|

$left$   $right$   
 $loc$

Since  $a[loc] > a[right]$ , interchange the two values and set  $loc = right$ .

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 25 | 10 | 36 | 18 | 27 | 45 |
|----|----|----|----|----|----|

$left$   $right$   
 $loc$

Since  $a[loc] < a[left]$ , interchange the values and set  $loc = left$ .

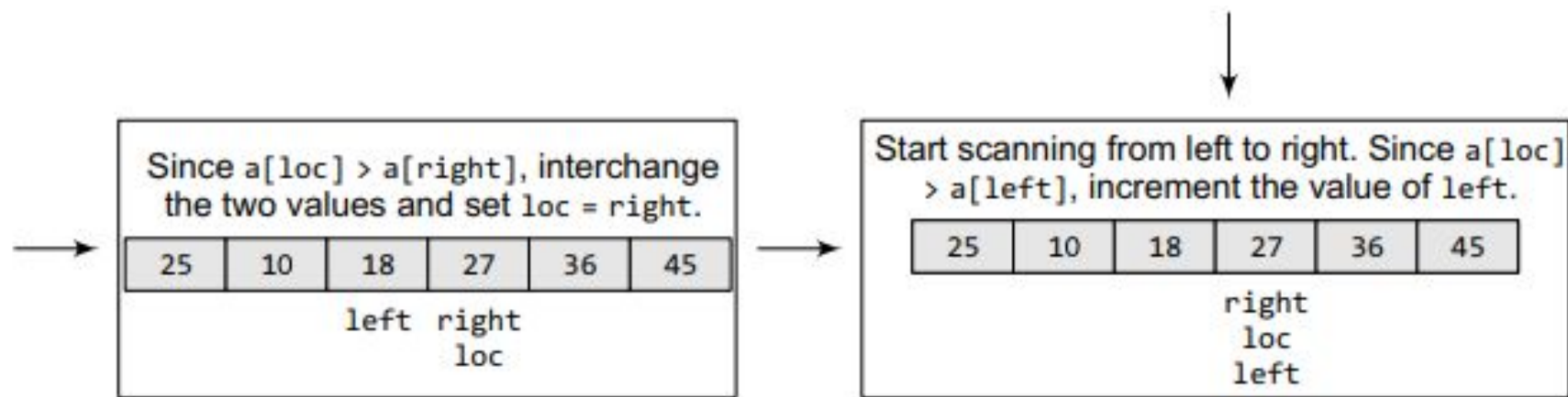
|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 25 | 10 | 27 | 18 | 36 | 45 |
|----|----|----|----|----|----|

$left$   $right$   
 $loc$

Scan from right to left. Since  $a[loc] < a[right]$ , decrement the value of  $right$ .

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 25 | 10 | 27 | 18 | 36 | 45 |
|----|----|----|----|----|----|

$left$   $right$   
 $loc$



Now  $left = loc$ , so the procedure terminates, as the pivot element (the first element of the array, that is, 27) is placed in its correct position. All the elements smaller than 27 are placed before it and those greater than 27 are placed after it.

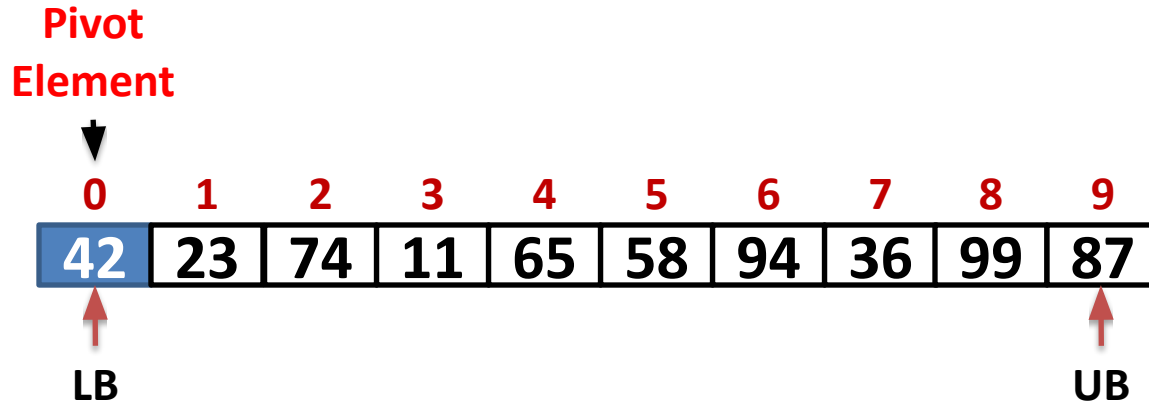
The left sub-array containing 25, 10, 18 and the right sub-array containing 36 and 45 are sorted in the same manner.

# Quick Sort

## Sort Following Array using Quick Sort Algorithm

We are considering **first element as pivot element**, so **Lower bound** is **First Index** and **Upper bound** is **Last Index**

We need to find our proper position of Pivot element in sorted array and perform same operations recursively for two sub array



# Quick Sort

FLAG  $\square$  true

IF LB < UB

Then

I  $\square$  LB

J  $\square$  UB + 1

KEY  $\square$  K[LB]

**Repeat While FLAG = true**

I  $\square$  I+1

**Repeat While** K[I] < KEY

I  $\square$  I + 1

J  $\square$  J - 1

**Repeat While** K[J] > KEY

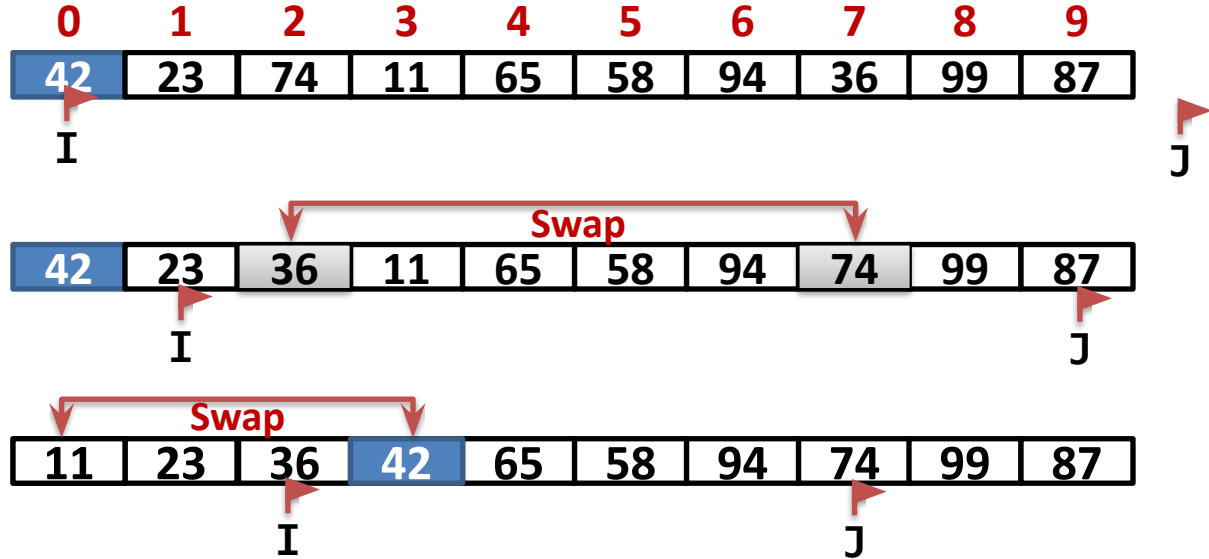
J  $\square$  J - 1

IF I < J

Then K[I]  $\square$  ---  $\square$  K[J]

Else FLAG  $\square$  FALSE

K[LB]  $\square$  ---  $\square$  K[J]



LB = 0, UB = 9

KEY = 42

I = 0

J = 10

FLAG = true

# Quick Sort

FLAG  $\square$  true

IF LB < UB

Then

I  $\square$  LB

J  $\square$  UB + 1

KEY  $\square$  K[LB]

**Repeat While FLAG = true**

I  $\square$  I+1

**Repeat While** K[I] < KEY

I  $\square$  I + 1

J  $\square$  J - 1

**Repeat While** K[J] > KEY

J  $\square$  J - 1

IF I < J

Then K[I]  $\square$  ---  $\square$  K[J]

Else FLAG  $\square$  FALSE

K[LB]  $\square$  ---  $\square$  K[J]

| LB |    |    | UB |    |    |    |    |    |    |  |
|----|----|----|----|----|----|----|----|----|----|--|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |  |
| 11 | 23 | 36 | 42 | 65 | 58 | 94 | 74 | 99 | 87 |  |

|    |    |    |
|----|----|----|
| 11 | 23 | 36 |
| I  |    | J  |

| LB |    |    | UB |    |    |    |    |    |    |  |
|----|----|----|----|----|----|----|----|----|----|--|
| 11 | 23 | 36 | 42 | 65 | 58 | 94 | 74 | 99 | 87 |  |

|    |    |   |
|----|----|---|
| 23 | 36 |   |
| I  |    | J |

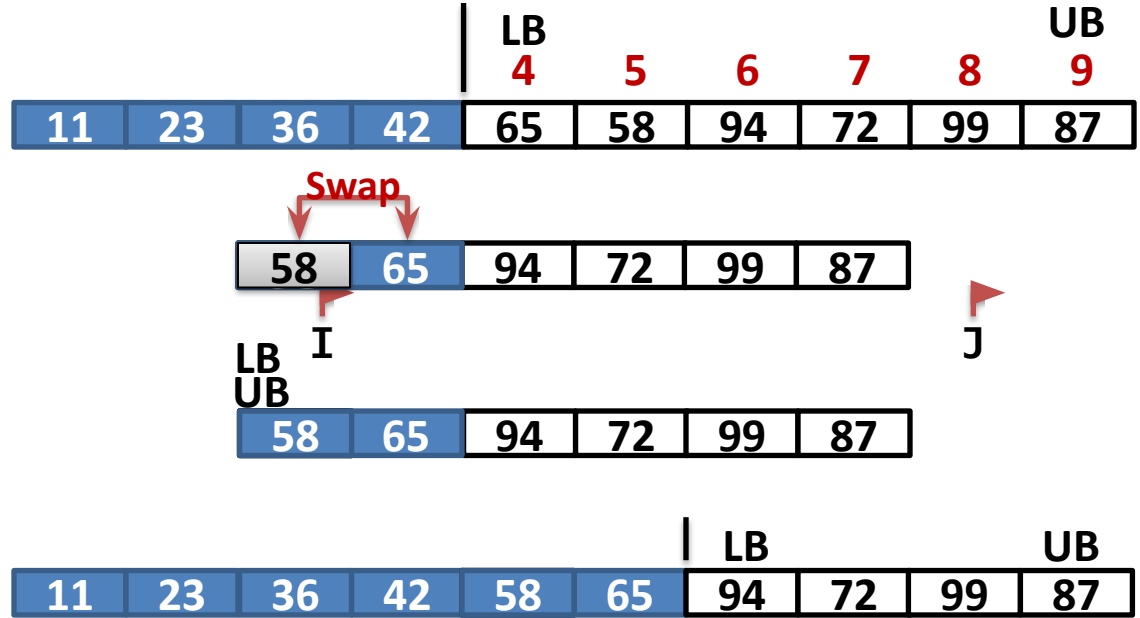
| LB |    |    | UB |    |    |    |    |    |    |  |
|----|----|----|----|----|----|----|----|----|----|--|
| 11 | 23 | 36 | 42 | 65 | 58 | 94 | 74 | 99 | 87 |  |

# Quick Sort

```

FLAG  $\square$  true
IF  LB < UB
Then
  I  $\square$  LB
  J  $\square$  UB + 1
  KEY  $\square$  K[LB]
  Repeat While FLAG = true
    I  $\square$  I+1
    Repeat While K[I] < KEY
      I  $\square$  I + 1
    J  $\square$  J - 1
    Repeat While K[J] > KEY
      J  $\square$  J - 1
    IF  I < J
    Then K[I]  $\square$  ---  $\square$  K[J]
    Else FLAG  $\square$  FALSE

  K[LB]  $\square$  ---  $\square$  K[J]
  
```

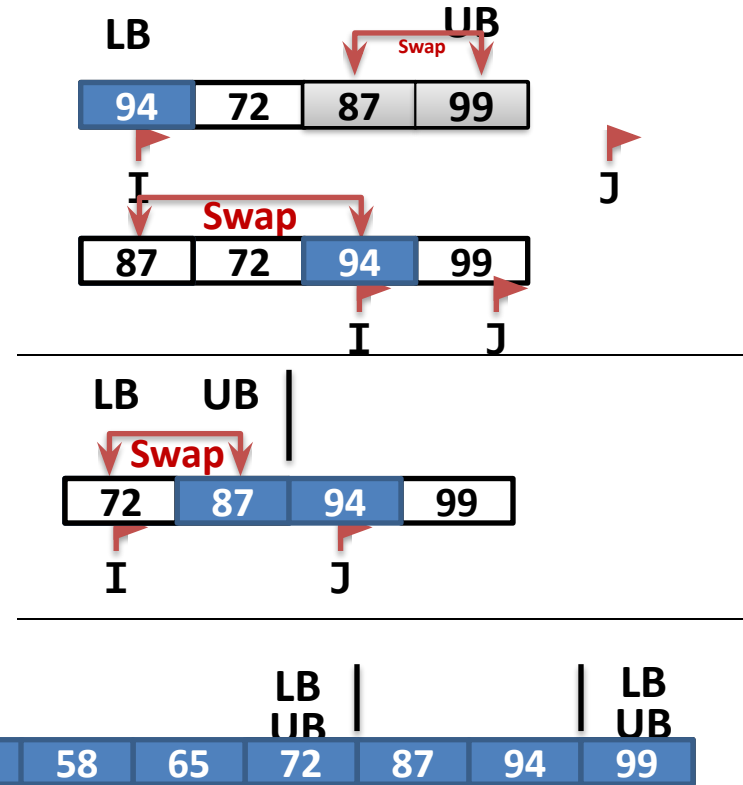


# Quick Sort

```

FLAG  $\square$  true
IF  LB < UB
Then
  I  $\square$  LB
  J  $\square$  UB + 1
  KEY  $\square$  K[LB]
  Repeat While FLAG = true
    I  $\square$  I+1
    Repeat While K[I] < KEY
      I  $\square$  I + 1
    J  $\square$  J - 1
    Repeat While K[J] > KEY
      J  $\square$  J - 1
    IF  I < J
    Then K[I]  $\square$  ---  $\square$  K[J]
    Else FLAG  $\square$  FALSE

  K[LB]  $\square$  ---  $\square$  K[J]
  
```



## Algorithm : Quick\_Sort(K, LB, UB)

```
1. [Initialize]
   FLAG  $\square$  true
2. [Perform Sort]
   IF LB < UB
   Then I  $\square$  LB
       J  $\square$  UB + 1
       KEY  $\square$  K[LB]
       Repeat While FLAG = true
           I  $\square$  I+1
           Repeat While K[I] < KEY
               I  $\square$  I + 1
           J  $\square$  J - 1
           Repeat While K[J] > KEY
               J  $\square$  J - 1
           IF I < J
           Then K[I]  $\square$  ---  $\square$  K[J]
           Else FLAG  $\square$  FALSE

       K[LB]  $\square$  ---  $\square$  K[J]
```

```
CALL QUICK_SORT(K, LB, J-1)
CALL QUICK_SORT(K, J+1, UB)
```

```
//CALL QUICK_SORT(K, LB, J-1)
```

```
3. [Finished]
   Return
```

## Comparison of different sorting techniques

| Analysis Type    | Bubble Sort | Insertion Sort | Merge Sort  | Quick Sort  |
|------------------|-------------|----------------|-------------|-------------|
| Best Case        | $O(n^2)$    | $O(n)$         | $O(\log n)$ | $O(\log n)$ |
| Average Case     | $O(n^2)$    | $O(n^2)$       | $O(\log n)$ | $O(\log n)$ |
| Worst Case       | $O(n^2)$    | $O(n^2)$       | $O(\log n)$ | $O(n^2)$    |
| Space Complexity | $O(1)$      | $O(1)$         | $O(n)$      | $O(\log n)$ |

# Usecase

- When to use bubble sort?
  - Bubble sort algorithm works well with large datasets where the items are almost sorted because it takes only one iteration to detect whether the list is sorted or not.
  - if the list is unsorted to a large extend then this algorithm holds good for collections of less number of elements.
  - Bubble sort is fastest on an extremely small or nearly sorted set of data.

# Usecase

- When to use insertion sort?
  - It is not advised to apply insertion sort when elements are in reverse order. In that case insertion sort takes maximum time to sort.
  - When the elements are sorted it takes the minimum time i.e  $O(N)$ . If the data is almost sorted or when the list is small as it has a complexity of  $O(N^2)$ . If the list is sorted a minimum number of elements will slide over to insert the element at its correct location i.e at sorted array part.
  - Algorithm is stable and it has fast running case when the list is nearly sorted.

# Usecase

- When to use merge sort?
  - Merge sort is used when the data structure doesn't support random access since it works with pure sequential access that is forward iterators, rather than random access iterators.
  - It is widely used for external sorting, where random access can be very, very expensive compared to sequential access.
  - It is used where it is known that the data is similar data.
  - Merge sort is fast in the case of a linked list because need for random access in linked list is low.

# Usecase

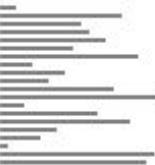
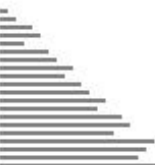
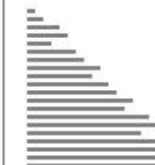
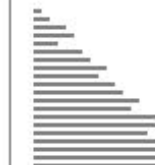
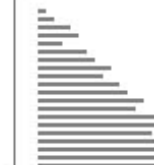
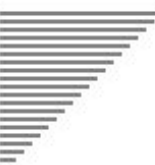
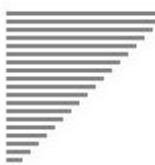
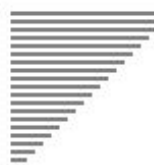
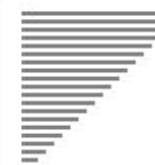
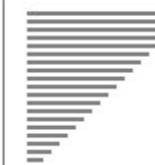
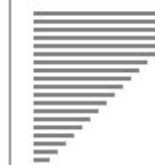
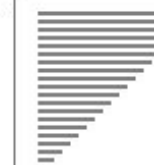
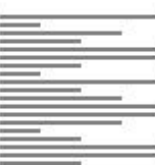
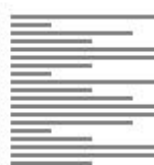
- When to use quick sort?
  - Quick sort is fastest, but it is not always  $O(N \log N)$ , as it becomes  $O(N^2)$  in worst cases.
  - Quicksort is probably more effective for datasets that fit in memory. For larger data sets it proves to be inefficient so algorithms like merge sort are preferred in that case.
  - Quick Sort is an in-place sort (i.e. it doesn't require any extra storage) so it is appropriate to use it for collections like arrays.

# Thank you

## Array Sorting Algorithms

| Algorithm             | Time Complexity     |                        |                   | Space Complexity |
|-----------------------|---------------------|------------------------|-------------------|------------------|
|                       | Best                | Average                | Worst             | Worst            |
| <u>Quicksort</u>      | $\Omega(n \log(n))$ | $\theta(n \log(n))$    | $O(n^2)$          | $O(\log(n))$     |
| <u>Mergesort</u>      | $\Omega(n \log(n))$ | $\theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |
| <u>Timsort</u>        | $\Omega(n)$         | $\theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |
| <u>Heapsort</u>       | $\Omega(n \log(n))$ | $\theta(n \log(n))$    | $O(n \log(n))$    | $O(1)$           |
| <u>Bubble Sort</u>    | $\Omega(n)$         | $\theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <u>Insertion Sort</u> | $\Omega(n)$         | $\theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <u>Selection Sort</u> | $\Omega(n^2)$       | $\theta(n^2)$          | $O(n^2)$          | $O(1)$           |
| <u>Tree Sort</u>      | $\Omega(n \log(n))$ | $\theta(n \log(n))$    | $O(n^2)$          | $O(n)$           |
| <u>Shell Sort</u>     | $\Omega(n \log(n))$ | $\theta(n(\log(n))^2)$ | $O(n(\log(n))^2)$ | $O(1)$           |
| <u>Bucket Sort</u>    | $\Omega(n+k)$       | $\theta(n+k)$          | $O(n^2)$          | $O(n)$           |
| <u>Radix Sort</u>     | $\Omega(nk)$        | $\theta(nk)$           | $O(nk)$           | $O(n+k)$         |
| <u>Counting Sort</u>  | $\Omega(n+k)$       | $\theta(n+k)$          | $O(n+k)$          | $O(k)$           |
| <u>Cubesort</u>       | $\Omega(n)$         | $\theta(n \log(n))$    | $O(n \log(n))$    | $O(n)$           |

|                |  Insertion |  Selection |  Bubble |  Shell |  Merge |  Heap |  Quick |  Quick3 |
|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
|  Random        |            |            |         |        |        |       |        |         |
|  Nearly Sorted |            |            |         |        |        |       |        |         |
|  Reversed      |            |            |         |        |        |       |        |         |
|  Few Unique    |            |            |         |        |        |       |        |         |

|                    | <br>Insertion | <br>Selection | <br>Bubble | <br>Shell | <br>Merge | <br>Heap | <br>Quick | <br>Quick3 |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <br>Random        |              |              |           |          |          |         |          |           |
| <br>Nearly Sorted |              |              |           |          |          |         |          |           |
| <br>Reversed      |              |              |           |          |          |         |          |           |
| <br>Few Unique    |              |              |           |          |          |         |          |           |



**Thank You**